CO1 ASSIGNMENT
EXPERIMENT – 2

LOGICAL, BIT-MANIPULATION AND SHIFT OPERATIONS USING 8-BIT AND 16-BIT DATA

Course / Subject	**Microprocessors and Microcontrollers (MPMC)**
Course Outcome	CO1
Bloom's Taxonomy Level	K3 – Apply
Work Knowledge Level	WK3 – Engineering Fundamentals
Simulator	8085 Simulator Software Kit
Student Name	B Akshaya
Register Number	192424336
Department	Artificial Intelligence and Data Science (AIDS)

This assignment covers bit masking and logical operations, bit setting and clearing, rotate and shift operations, complement operations, and bit-level counting.

INDEX

S. No.	Contents	Section
1	Aim, objectives, outcomes and requirements	1
2	Theory and instruction set	2
3	Sub-experiment A: Bit masking and logical operations	3
4	Sub-experiment B: Bit setting, clearing, shift and rotate	4
5	Combined output table, result and viva-voce	5

1. AIM

To develop, execute and verify 8085 assembly-language programs that perform AND, OR, XOR, one's complement, two's complement, bit masking, bit setting, bit clearing, bit toggling, bit testing, rotate and shift operations on 8-bit and 16-bit data stored in memory.

Objectives

- Perform AND, OR and XOR operations on two 8-bit operands.
- Apply masks to select upper and lower nibbles and to set, clear, toggle and test individual bits.
- Compute one's and two's complements of 8-bit and 16-bit values.
- Execute rotate, logical shift and arithmetic shift operations and analyze the carry flag.
- Count the number of set bits in an 8-bit value and verify results using manual binary calculations.

Expected outcomes

- Obtain correct bitwise logical results and complement values.
- Choose masks that preserve, set, clear, toggle or test specified bits.
- Interpret carry and condition flags after logical, rotate and shift instructions.
- Verify all outputs against manual calculations with zero deviation.

Software and hardware required

- Computer/laptop with the 8085 Simulator Software Kit.
- 8085 processor trainer kit, keyboard and power cord, if available.

2. FUNDAMENTAL THEORY

The 8085 is an 8-bit microprocessor. Logical instructions operate on the accumulator and an operand. ANA performs bitwise AND, ORA performs bitwise OR, and XRA performs bitwise exclusive OR. CMA complements every bit of the accumulator. Adding one to the complemented value produces the two's complement.

Bit masking uses a constant called a mask. AND with 0 clears a bit and AND with 1 preserves it. OR with 1 sets a bit, while XOR with 1 toggles a bit. Rotates move bits around within the accumulator or through

the carry flag. Logical shifts insert zero into the vacated bit position, while arithmetic right shift preserves the sign bit.

Important instructions

Instruction	Purpose
ANA / ORA / XRA	Perform bitwise AND, OR and XOR.
ANI / ORI / XRI	Perform AND, OR and XOR with an immediate mask.
CMA	Complement all bits in the accumulator.
INR A	Increment A by one; used after CMA for two's complement.
RLC / RRC	Rotate accumulator left/right; outgoing bit is copied to CY.
RAL / RAR	Rotate accumulator left/right through carry.
DAD H	Add HL to itself, producing a 16-bit left shift.
LHLD / SHLD	Load/store a 16-bit value in memory, low byte first.
PUSH PSW / POP	Save/restore accumulator and flag status using the stack.
JZ / JNC / JNZ	Conditional jumps based on flags.

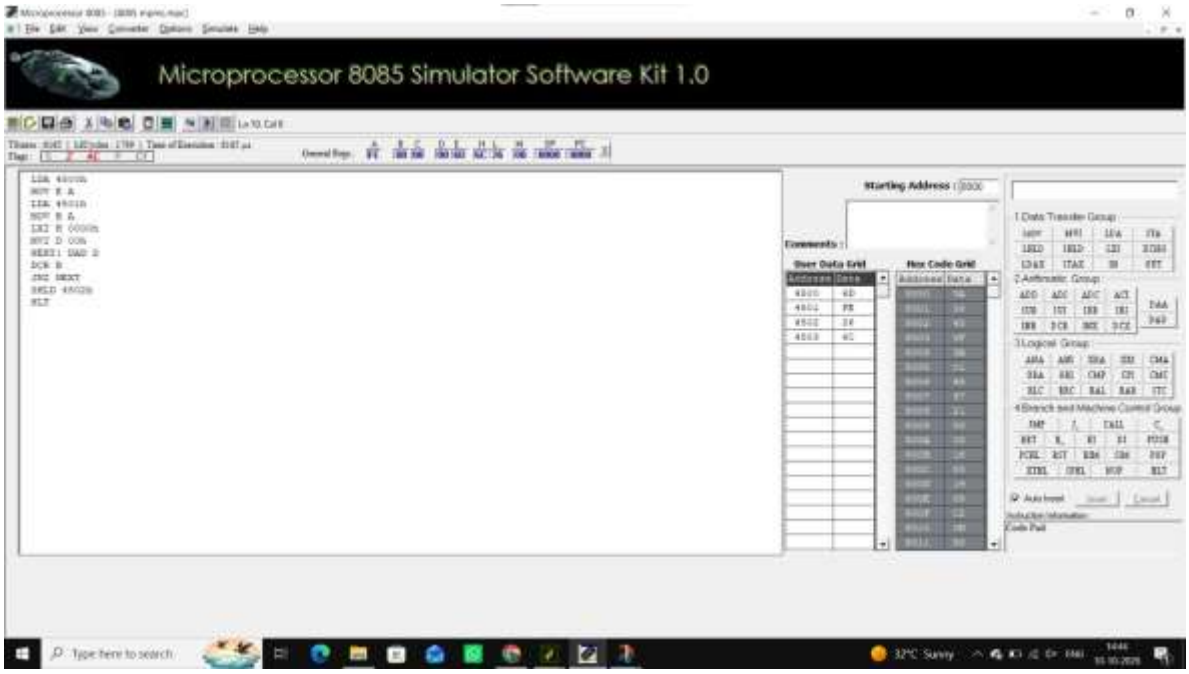
Flag notes

The 8085 condition flags are Sign (S), Zero (Z), Auxiliary Carry (AC), Parity (P) and Carry (CY). ANA/ANI clear CY and set AC; ORA/ORI and XRA/XRI clear CY and AC. CMA does not affect flags. RLC/RRC place the rotated-out bit in CY. RAL/RAR rotate through CY, so CY should be cleared before using them for a logical shift. For arithmetic right shift, the original sign bit is copied to CY before RAR.

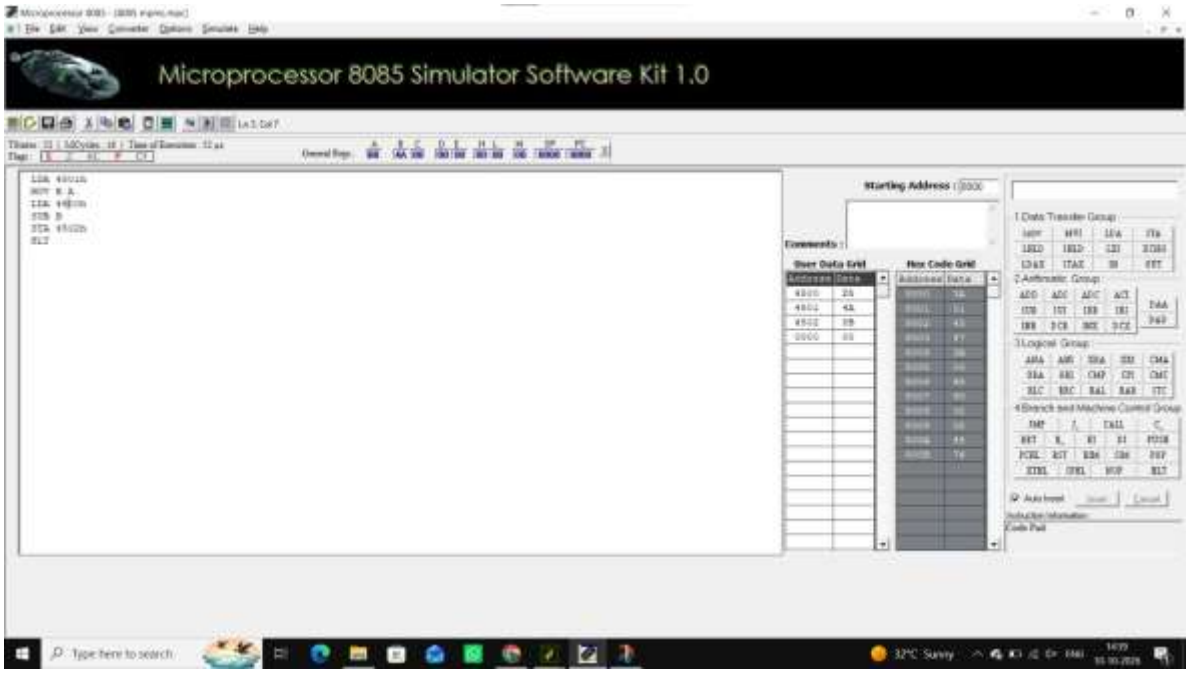
Memory convention: 16-bit values are stored in little-endian order. For example, 1234H is stored as 34H at the lower address and 12H at the next address.

3. SUB-EXPERIMENT A — VERIFICATION OF BIT MASKING OPERATIONS

Actual SIM8085 screenshot 1 — Program A execution



Actual SIM8085 screenshot 2 — Program A execution



Aim

To perform AND, OR, XOR, nibble masking, one's complement and two's complement operations on 8-bit and 16-bit data and verify the results using the 8085 simulator.

Input values

Input	Memory address	Value
Operand X	4500H	A7H
Operand Y	4501H	3CH
16-bit word W	450AH–450BH	1234H (34H, 12H)

Theory and manual calculations

A7H = 1010 0111 and 3CH = 0011 1100. AND retains bits set in both operands: A7H AND 3CH = 0010 0100 = 24H. OR sets a bit if either operand contains 1: A7H OR 3CH = BFH. XOR sets a bit when the operands differ: A7H XOR 3CH = 9BH.

The lower nibble is extracted using 0FH: A7H AND 0FH = 07H. The upper nibble is extracted using F0H: A7H AND F0H = A0H. CMA produces the one's complement of A7H, which is 58H; adding one gives the two's complement 59H. For W = 1234H, the one's complement is EDCBH and the two's complement is EDCCH.

Algorithm

- Initialize the stack pointer and load X and Y from memory using HL.
- Perform ANA B and store the result and flag byte.
- Reload X and calculate OR and XOR with Y.
- Use ANI 0FH and ANI F0H to extract the lower and upper nibbles.
- Use CMA for one's complement and INR A to calculate two's complement.
- Load the 16-bit word W, complement both bytes, store the one's complement, increment HL and store the two's complement.
- Stop execution and inspect the output memory locations.

Assembly-language program (starting address 4100H)

```
LXI SP,47FFH
LXI H,4500H
MOV A,M
INX H
MOV B,M
ANA B
PUSH PSW
STA 4502H
POP H
MOV A,L
STA 4509H
LDA 4500H
ORA B
STA 4503H
LDA 4500H
XRA B
STA 4504H
LDA 4500H
ANI 0FH
STA 4505H
LDA 4500H
ANI 0F0H
STA 4506H
LDA 4500H
CMA
STA 4507H
INR A
STA 4508H
LHLD 450AH
MOV A,L
CMA
MOV L,A
MOV A,H
```

CMA

MOV H,A

SHLD 450CH

INX H

SHLD 450EH

HLT

Procedure

- Open the 8085 simulator and enter Program A.
- Set the starting address to 4100H and assemble/convert the program.
- Enter A7H at 4500H, 3CH at 4501H, and W = 1234H at 450AH–450BH in low-byte-first order.
- Execute the program using GO/EXEC.
- Read each output memory location and compare it with the theoretical value.
- Record the actual flag byte at 4509H and calculate deviation.

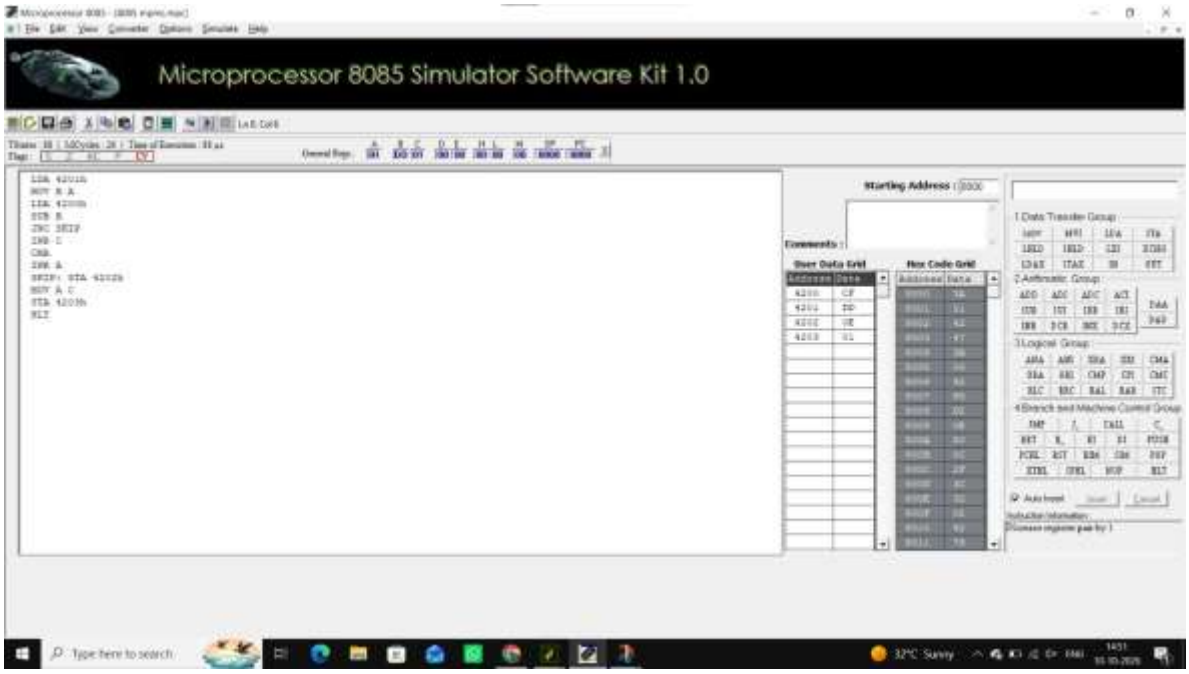
Expected output table

Output parameter	Address	Expected value
X AND Y	4502H	24H
Flags after ANA	4509H	14H (S=0, Z=0, AC=1, P=1, CY=0)
X OR Y	4503H	BFH
X XOR Y	4504H	9BH
Lower nibble	4505H	07H
Upper nibble	4506H	A0H
One’s complement of X	4507H	58H
Two’s complement of X	4508H	59H
One’s complement of W	450CH–450DH	EDCBH
Two’s complement of W	450EH–450FH	EDCCH

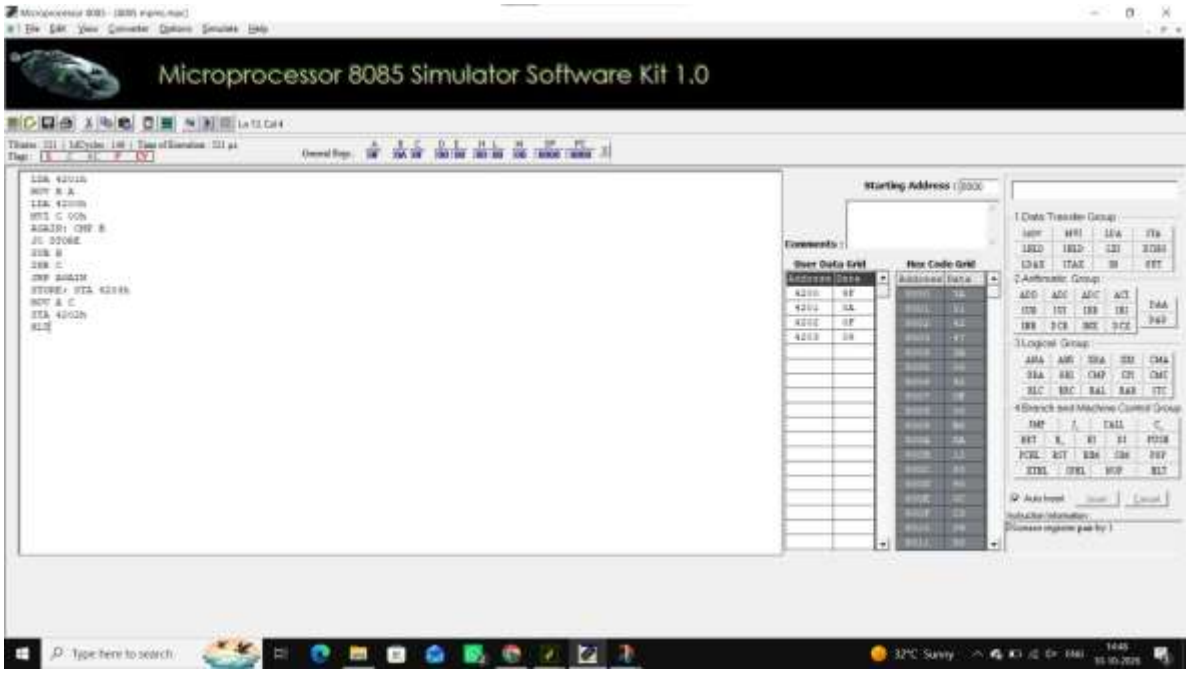
Simulator screenshot — Sub-experiment A

4. SUB-EXPERIMENT B — BIT SETTING, CLEARING, SHIFT AND ROTATE

Actual SIM8085 screenshot 1 — Program B execution



Actual SIM8085 screenshot 2 — Program B execution



Aim

To set, clear, toggle and test selected bits of an 8-bit value, perform rotate and shift operations, count set bits, and shift a 16-bit value using 8085 instructions.

Input values

Input	Memory address	Value
Data byte D	4510H	CAH
16-bit word	4520H–4521H	9234H (34H, 92H)

Theory and manual calculations

CAH = 1100 1010. ORI 81H sets bits 7 and 0, producing CBH. ANI F5H clears bits 3 and 1, producing C0H. XRI 0FH toggles the lower nibble, producing C5H. Bit 6 is set in CAH, so the bit-status result is 01H.

RLC CAH produces 95H with CY=1; RRC CAH produces 65H with CY=0. Logical shift left produces 94H with CY=1, and logical shift right produces 65H with CY=0. Arithmetic right shift preserves the sign bit and produces E5H. CAH contains four set bits. Shifting 9234H left by one bit produces 2468H with CY=1.

Algorithm

- Load D and set bits 7 and 0 using ORI 81H; store the result.
- Reload D, clear bits 3 and 1 with ANI F5H, then toggle the lower nibble using XRI 0FH.
- Test bit 6 with ANI 40H and store 01H if set, otherwise 00H.
- Perform RLC and RRC and store both outputs.
- Clear CY before RAL/RAR for logical shifts; copy the sign bit to CY before RAR for arithmetic right shift.
- Rotate the original byte eight times and count carry occurrences to count the set bits.
- Load the 16-bit word and use DAD H to shift it left by one bit; store the result and halt.

Assembly-language program (starting address 4200H)

```
LDA 4510H
ORI 81H
STA 4511H
LDA 4510H
ANI 0F5H
STA 4512H
LDA 4510H
XRI 0FH
STA 4513H
```

```
MVI C,00H
LDA 4510H
ANI 40H
JZ NEXT
INR C
NEXT: MOV A,C
STA 4514H
LDA 4510H
RLC
STA 4515H
LDA 4510H
RRC
STA 4516H
LDA 4510H
ORA A
RAL
STA 4517H
LDA 4510H
ORA A
RAR
STA 4518H
LDA 4510H
MOV B,A
RAL
MOV A,B
RAR
STA 4519H
LDA 4510H
MVI B,08H
MVI C,00H
CNT: RLC
JNC SKIP
INR C
SKIP: DCR B
```

JNZ CNT

MOV A,C

STA 451AH

LHLD 4520H

DAD H

SHLD 4522H

HLT

Procedure

- Open the simulator and enter Program B.
- Set the starting address to 4200H and assemble/convert the program.
- Enter CAH at 4510H and 9234H at 4520H–4521H, low byte first.
- Execute the program using GO/EXEC.
- Read the results from 4511H–451AH and the 16-bit result from 4522H–4523H.
- Compare each output with the expected table and record actual results.

Expected output table

Output parameter	Address	Expected value
Set bits 7 and 0	4511H	CBH
Clear bits 3 and 1	4512H	C0H
Toggle lower nibble	4513H	C5H
Bit-6 status	4514H	01H
RLC	4515H	95H; CY=1
RRC	4516H	65H; CY=0
Logical shift left	4517H	94H; CY=1
Logical shift right	4518H	65H; CY=0
Arithmetic shift right	4519H	E5H
Number of 1-bits	451AH	04H
16-bit shift left	4522H–4523H	2468H; CY=1

Simulator screenshot — Sub-experiment B

5. COMBINED OUTPUT / OBSERVATION TABLE

No.	Output	Expected value
1	X AND Y (4502H)	24H
2	Flags after ANA (4509H)	14H
3	X OR Y (4503H)	BFH
4	X XOR Y (4504H)	9BH
5	Lower / upper nibble (4505H, 4506H)	07H, A0H
6	One's / two's complement of X (4507H, 4508H)	58H, 59H
7	One's / two's complement of W (450CH, 450EH)	EDCBH, EDCCH
8	Set / clear bits (4511H, 4512H)	CBH, C0H
9	Toggle / bit-6 status (4513H, 4514H)	C5H, 01H
10	RLC / RRC (4515H, 4516H)	95H (CY=1), 65H (CY=0)
11	Logical left / right shift (4517H, 4518H)	94H (CY=1), 65H (CY=0)
12	Arithmetic shift / count ones (4519H, 451AH)	E5H, 04H
13	16-bit shift left (4522H)	2468H (CY=1)

Formulae used

- AND: a result bit is 1 only when both input bits are 1.
- OR: a result bit is 1 when either input bit is 1. XOR: a result bit is 1 when the input bits differ.
- One's complement: NOT X = FFH − X. Two's complement: −X = (NOT X) + 1.
- Deviation = |Observed value − Theoretical value|; the expected deviation is zero.
- Division is not part of this experiment; shift results are verified bit by bit and carry is recorded where applicable.

6. RESULT

The experiment demonstrates logical operations, bit masking, one's and two's complement, bit setting, clearing, toggling and testing, rotate operations, logical and arithmetic shifts, and set-bit counting using the 8085 simulator. The expected output values are calculated manually and should be compared with actual memory values after execution.

Precautions

- Use the specified program start addresses: 4100H for Program A and 4200H for Program B.
- Enter all input values at the correct memory addresses; store 16-bit data in little-endian order.
- Clear CY before using RAL/RAR as logical shifts.
- Preserve the original input value when comparing multiple operations.
- Record actual simulator output and flag values; do not claim an observation that was not performed.